



Vision Transformer CIFAR10

데이터셋 추론하기

JD-edu

© 2025 JD-edu. All rights reserved.



Fine-tuning된 모델 확보

JD-eud에서 미리 fine-tuning작업된 모델을 구글 드라이브에서 얻기

<https://drive.google.com/file/d/1LRfvr33F8my0XKCF4J7ihw23aKueLc24/view?usp=sharing>

주피터노트북 혹은 로컬 PC에서 Fine tuning을
진행해서 직접 확보하기

Fine_tuning_ViT_CIFAR_10.py

Fine_tuning_ViT_CIFAR_10.ipynb

디스크 이미지를 추론하기 - 모델 로딩

```
import torch
from PIL import Image
from transformers
import ViTForImageClassification, ViTImageProcessor

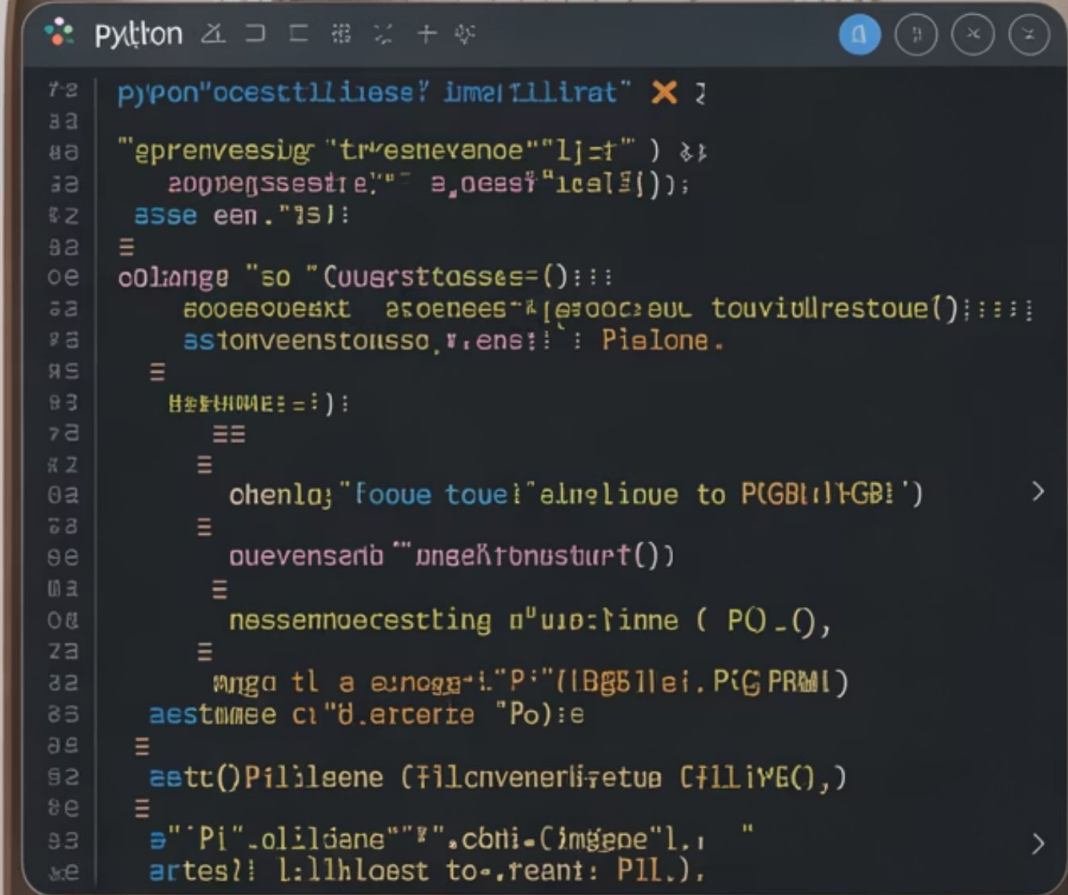
MODEL_PATH = "./" # 모델이 저장된 폴더 경로

try:
    processor = ViTImageProcessor.from_pretrained(MODEL_PATH)
    model = ViTForImageClassification.from_pretrained(MODEL_PATH)
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model.to(device) model.eval() # 추론 모드로 설정
    print(f"Model load completed. Device {device}")
except Exception as e:
    print("Model load error!!") exit()
```

디스크 이미지 추론하기 - 이미지 전처리

```
IMAGE_FILE_PATH = "plane4.png"

try:
    image = Image.open(IMAGE_FILE_PATH).convert("RGB")
except FileNotFoundError:
    print(f"오류: 이미지 파일 '{IMAGE_FILE_PATH}'을 찾을 수 없음.")
    exit()
```





디스크 이미지 추론하기 - 이미지 해석

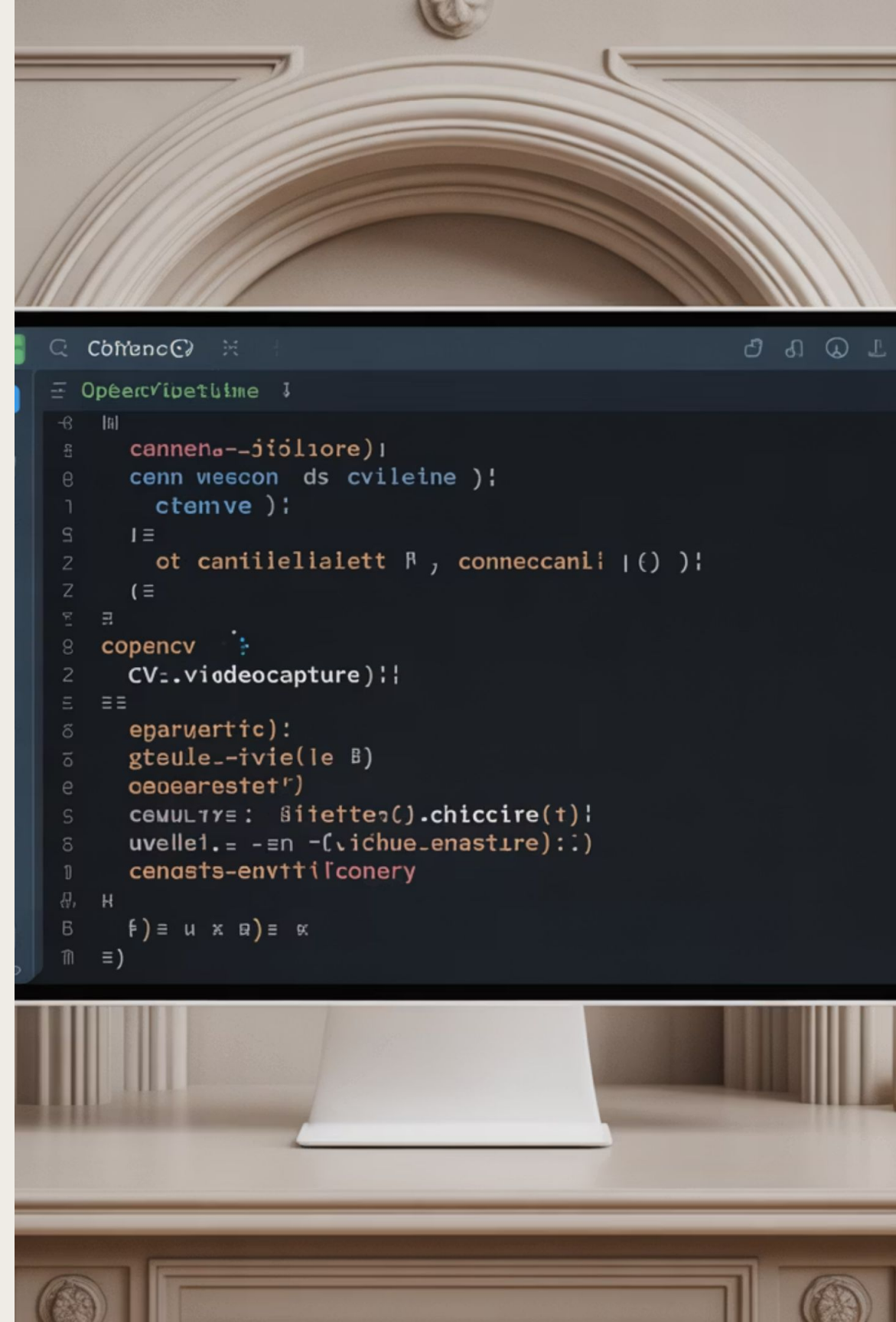
```
inputs = processor(images=image, return_tensors="pt").to(device)
with torch.no_grad():
    outputs = model(**inputs)
logits = outputs.logit
spredicted_class_idx = logits.argmax(-1).item()
predicted_label = model.config.id2label[predicted_class_idx]
print("\n--- 추론 결과 ---")
print(f"예측된 클래스 인덱스: {predicted_class_idx}")
print(f"예측된 클래스 이름: **{predicted_label}**")
```

카메라 스트림 추론하기 - 모델로딩

```
import torch
from PIL import Image
from transformers import ViTForImageClassification, ViTImageProcessor
import cv2 # 웹캠 사용을 위한 OpenCV 라이브러리
import numpy as np # OpenCV와 PIL 이미지 변환을 위한 NumPy
MODEL_PATH = "./"
try:
    processor = ViTImageProcessor.from_pretrained(MODEL_PATH)
    model = ViTForImageClassification.from_pretrained(MODEL_PATH) # CPU 환경을 기본으로 사용합니다.
    device = torch.device("cpu")
    model.to(device)
    model.eval() # 추론 모드로 설정 print(f" 모델 로드 완료. 현재 Device: {device}")
except Exception as e:
    print(f"모델 로드 중 오류 발생: {e}")
    print("폴더 경로와 파일 존재 여부를 확인해주세요.") exit()
```

카메라 스트림 추론하기 - 카메라

```
cap = cv2.VideoCapture(0)
if not cap.isOpened():
    print("오류: 웹캠을 열 수 없습니다. 카메라 연결을 확인하세요.")
    exit()
```



카메라 라이브 스트림 추론 실행



```
while True:
    # 프레임 읽기
    ret, frame = cap.read()
    if not ret:
        print("프레임을 읽을 수 없습니다.")
        break

    frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    image = Image.fromarray(frame_rgb)
    # Numpy 배열을 PIL 이미지 객체 변환 ViT 프로세서가 선호
    inputs = processor(images=image, return_tensors="pt").to(device)
    # ViT 프로세서 전처리 및 텐서 생성
    with torch.no_grad(): # 추론 실행
        outputs = model(**inputs)
```




```
logits = outputs.logits

predicted_class_idx = logits.argmax(-1).item()

# 클래스 이름으로 변환

predicted_label = model.config.id2label[predicted_class_idx]

# 결과 화면에 표시, 예측 결과를 OpenCV 프레임에 오버레이

text = f"Predicted: {predicted_label}"

# (프레임, 텍스트, 시작 위치, 폰트, 크기, 색상, 두께)

cv2.putText(frame, text, (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2, cv2.LINE_AA)
```



카메라 라이브 스트림 마무리

```
cv2.imshow('ViT Live Classification', frame)
# 종료 조건 'q' 키 또는 ESC 키를 누르면 루프 종료
key = cv2.waitKey(1) & 0xFF
if key == ord('q') or key == 27:
    break

cap.release()
cv2.destroyAllWindows()
print("라이브 스트림 종료 및 자원 해제 완료.")
```